Tools in the Context Window: An Open Benchmark for MCP versus CLI Tool Use Across Agent Scaffoldings

Every number and figure in this paper is generated from `results/final.jsonl` by `bench/fill_paper.py`; nothing is hand-entered.

Abstract

An agent needs a way to act on the world, and there are two dominant ways to give it one. The Model Context Protocol publishes a catalogue of typed tools that the host injects into the model's context. A command-line tool publishes nothing: the agent writes a command string and reads what comes back. Practitioners report that the first approach is far more expensive in tokens than the second, with figures ranging from a modest multiple to more than an order of magnitude, but these reports are single-setup anecdotes and none of them can be re-run.

We present an open benchmark that measures the difference on one fixed task across four agent scaffoldings — Claude Code, OpenAI Codex, qwen-code and pi — and nine models spanning hosted frontier systems and locally-served open weights. Each scaffolding runs the same workflow twice, once with a GitHub MCP server attached and once with its native shell and the `gh` command-line tool, in its default configuration. We record initial context size, total and cached tokens, the number of tool calls and which tools were called, tool-call success, the percentage of the task completed against a rubric, and a theoretical cost computed from one public price list applied uniformly to every cell.

Across 19 comparable cells the MCP arm costs a median **1.71×** the tokens of the CLI arm, but the ratio ranges from **0.27×** to **7.41×** — a 27-fold spread in the ratio itself. The same protocol, task and fixture can make MCP either substantially cheaper or substantially more expensive than a command line, depending on which scaffolding and model run underneath. We therefore report a distribution rather than a ratio, and find that the choice of scaffolding moves total cost further than the choice of surface does.

The harness, the pinned fixture repository, the task, the scoring rubric and the raw run logs are public, so every number here can be re-derived and every claim can be attacked with the same instrument that produced it.

1. Introduction

The question this paper measures is narrow and practical: when you give an agent a capability, what does the delivery mechanism cost you?

Two mechanisms dominate. Under the Model Context Protocol, a server advertises a catalogue of tools with names, descriptions and typed parameter schemas, and the host places that catalogue in the model's context so the model can select from it. Under the command-line approach, the agent already has a shell, and a capability arrives as knowledge of which commands to run — from the model's training, from a short skill document, or from a `--help` invocation the agent issues itself.

The mechanisms differ in where the description of the capability lives. MCP puts it in the context window, priced per token, on every request of the session. A CLI puts it in the filesystem, priced at nothing, and the agent pays only for the command it writes and the output it reads back.

That asymmetry is widely asserted. A matched-task comparison reports roughly 35× more tokens through MCP than through a CLI [MindStudio 2026]. A single GitHub query is measured at ~1,365 tokens via `gh` against ~44,026 via the corresponding MCP server [Vensas 2026]. A GitHub MCP server exposing 93 tools is reported to add ~55,000 tokens of registry overhead at session start [Reinhard 2026]. Practitioner write-ups put smaller catalogues at 13,700 and 18,000 tokens and observe that "7–9% of context is gone before you start" [Zechner 2026].

These numbers disagree with each other by more than an order of magnitude, and the disagreement is not surprising: each was measured on one scaffolding, with one model, on one task, and none ships a harness. A practitioner deciding how to build an agent cannot tell from this literature whether the penalty is 1.2× or 35×, nor what it depends on.

We think the disagreement is itself the finding, and that it is explained by variables the reports do not hold fixed. This paper's contribution is therefore not a single ratio. It is **an instrument**: a task, a rubric, a set of adapters that normalise four incompatible telemetry formats, and a published set of runs against which any of our claims can be checked.

Contributions

1. **An open, re-runnable benchmark** for MCP-versus-CLI tool use, spanning four scaffoldings and nine models, with a pinned fixture so ground truth cannot drift.
2. **A measurement across scaffoldings rather than within one**, which shows that the choice of scaffolding moves the result by more than the choice of surface.
3. **A set of measurement hazards**, each found by producing a wrong answer first, that any replication must control or silently mis-measure (§4).
4. **Raw run logs and a tool register** — not just how many tools were called, but which — so the mechanism behind the token difference is inspectable.

2. Related work

Practitioner measurements. The claim that MCP is expensive in tokens comes almost entirely from practitioner write-ups rather than from the literature, and they disagree by more than an order of magnitude. A matched-task comparison puts MCP at roughly 35× a CLI equivalent and reports task-completion reliability falling from 100% to 72% on harder scenarios [MindStudio 2026]. A single GitHub language query is measured at ~1,365 tokens through `gh` against ~44,026 through the matching MCP server [Vensas 2026]. A GitHub MCP server exposing 93 tools is reported to add ~55,000 tokens of registry overhead at session start against ~200 for the CLI equivalent [Reinhard 2026]. Smaller catalogues are put at 13,700 tokens for a browser-automation server and 18,000 for a developer-tools server, with the observation that "7–9% of context is gone before you start" [Zechner 2026].

Each of these is a single configuration measured once, and none publishes a harness. They are valuable as existence proofs and unusable as a basis for a decision, because none of them isolates which of the several plausible causes — catalogue size, schema verbosity, result verbosity, scaffolding overhead, model competence — produced the number.

The protocol and its own guidance. The Model Context Protocol specifies tools as named operations with descriptions, typed inputs and metadata exposed to the model [MCP 2025]. Anthropic's engineering guidance acknowledges that connecting many servers accumulates enough tool context to motivate code-execution patterns that reduce it [Anthropic 2026], which is a concession that

the overhead is real and a suggestion that it is addressable by configuration rather than intrinsic to the protocol.

Progressive disclosure of tools. The idea that tool descriptions should be fetched on demand rather than declared upfront appears both in practitioner writing and, as we found, already implemented in a shipping agent CLI: qwen-code declares deferred tool schemas upfront only when they fit inside a context-window budget and otherwise loads them through a search tool. That a mainstream scaffolding has independently converged on the mitigation is itself evidence about the size of the problem — and it is a confound for any measurement that does not pin the setting (§4.3).

Agent benchmarks. Existing agent benchmarks largely measure task success — whether the agent solves the problem — with cost as a secondary reporting convenience where it appears at all. This benchmark inverts that: the task is deliberately easy enough that most capable models complete it, so that the measurement is dominated by *how much it cost to complete* rather than by whether it was completed. Completion is retained as a rubric percentage precisely because weak models fail partially and that gradation carries information about the surface.

What is missing, and what this adds. No published work we are aware of measures the same task across multiple scaffoldings holding the surface constant, which is what isolates the scaffolding's own contribution from the protocol's. Our results suggest that contribution is large enough to dominate the comparison the literature is actually arguing about.

3. Method

3.1 The task

One workflow, run against a frozen snapshot of a real teaching repository (`Lamb-Project/aawd-e1-fixture`, tag `fixture-v1`). The agent is asked five questions and must answer all five in one numbered list:

1. the repository's default branch;
2. the top-level directory names;
3. the first heading of `README.md`;
4. the git tags that exist;
5. how many files are in the `week-01` directory tree.

Each item is checkable independently against ground truth, so a run is scored as a **percentage of the task completed** rather than pass or fail. Weak models are expected to fail partially, and where they fail is data.

Items 1–4 are lookups of increasing depth. **Item 5 is the discriminating one.** A shell can compute the answer server-side and return a single integer. A tool catalogue without an aggregate operation must list the directory, then list each subdirectory, and count by reasoning over every listing it has pulled into context. The task therefore contains one item where the two surfaces are structurally unequal, by design, because that inequality is the thing under study.

3.2 Arms

Each scaffolding/model pair runs twice:

- **MCP arm** — the official `github/github-mcp-server` (v1.8.0) attached over stdio in its default toolset configuration.

- **CLI arm** — the scaffolding's native shell, with the authenticated `gh` CLI available on `PATH` and no MCP server attached.

Every scaffolding is run **in its default configuration**. We do not prune tool registries, trim built-ins or tune prompts. The question is what a practitioner gets when they connect a server and start working, not what a tuned harness can be made to do.

Thinking or extended reasoning is disabled, or set to the lowest available setting, on every scaffolding. Reasoning tokens are a large and separately-varying cost that would swamp the effect under study.

3.3 Matrix

Scaffolding	Models	MCP arm	CLI arm
Claude Code	sonnet-5, opus-5, fable-5	✅	✅
Codex	gpt-5.6 sol/luna/terra, glm-5.2, qwen3.5:122b, qwen3.6:27b	✅	✅
qwen-code	same six	✅	✅
pi	same six	⛔ void	✅

Two cells in this table are structural rather than incidental. **pi ships no MCP client** — no flag, no configuration key, no mention in its help output — so its MCP cells are reported void. A void cell is not a zero: "this scaffolding cannot do MCP" and "MCP costs this scaffolding nothing" are opposite claims, and a table that renders both as an empty cell asserts the wrong one. **Claude Code reaches only Anthropic models**, having no custom-endpoint configuration, so it is not run against the open-weights models.

3.4 Measurements

Per run: initial (first-request) prompt tokens; total input and output tokens; cached tokens and cache-hit rate; tool-call count; **the register of which tools were called**; tool-call success ratio; task completion percentage; theoretical cost; and wall-clock time.

The four scaffoldings report telemetry in four incompatible formats, three of them undocumented. We established each by running the scaffolding and inspecting its output rather than by reading its documentation, and the adapters record a field as *absent* rather than substituting zero when a scaffolding does not report it — a missing cache figure and a genuine zero support opposite conclusions.

3.5 Cost model

Cost is **theoretical**, computed by applying OpenRouter's public per-token list prices uniformly to every cell, including cells served on local hardware.

This is deliberate. Half these models run on a machine in the room, where marginal cost is electricity, and the rest bill through different arrangements — a subscription, a metered key. Comparing invoices would compare accounting arrangements rather than workloads. One public, dated price

list applied uniformly gives a figure that is comparable across the matrix and reproducible by anyone.

Cached input is billed as input, with no discount assumed, because providers discount cached reads at different rates and local inference has no billing concept at all. Cache-hit rate is reported as its own dimension, so a reader who wants to apply a particular vendor's discount can.

Real money spent is recorded separately and never folded into this figure.

3.6 What wall-clock time can and cannot say

We record wall time and we do not compare it across the matrix. Locally-served models run on one machine with one request in flight; hosted models run on provider infrastructure with unknown batching. A timing comparison between them measures the hardware, not the surface. Within a single model, timings are also sensitive to cache state (§4.4). We publish the numbers because they are cheap to record and occasionally diagnostic, and we draw no conclusions from them.

4. Measurement hazards

Every hazard in this section was found by getting a wrong answer first. We report them because each produces *plausible* numbers, which is the dangerous kind of wrong, and because a replication that does not control them will not reproduce our results.

4.1 An MCP arm that silently is not one

Two of the four scaffoldings started with **no MCP tools attached and reported no error**. qwen-code leaves an unapproved server in a pending state and runs anyway; Codex ignored an inline configuration override. In both cases the agent fell back to its shell, completed the task, and produced a clean set of numbers that would have been recorded as an MCP measurement.

The harness therefore asserts, after every MCP run, that at least one MCP tool was actually called, and voids the cell otherwise.

4.2 A scaffolding reporting success on a failed task

In one run the scaffolding's own result event reported `subtype: "success"` and `is_error: false` for an answer that was incomplete — the agent was still counting files when it stopped. A harness that trusts the scaffolding's status would score it as a pass.

Completion is therefore scored only by the rubric, against ground truth, and never from the scaffolding's self-report.

4.3 Configuration that silently changes what is in context

Two settings in one scaffolding change the tool surface without changing anything the operator would think to record:

- a **tool-search budget** expressed as a percentage of the context window, which switches between declaring tool schemas upfront and fetching them on demand depending on whether they fit — so the same server produces different context costs at different registry sizes;
- an **approval mode**, which in a headless session does not merely block risky tools but **never registers them**, removing them from the model's context entirely. At the default setting the shell tool is absent, and a CLI arm cannot exist.

Both must be pinned and reported. Neither is something a casual replication would think to state.

4.4 Cache state that leaks between runs

The local inference server persists its KV cache to disk across runs. Whichever arm runs after a similar one therefore starts warm and appears dramatically cheaper and faster. In an early sweep the fastest run of the set was fastest only because of what had run before it.

Runs are interleaved by arm and the local cache is cleared between cells.

4.5 Reasoning that does not terminate

With extended thinking enabled, the locally-served GLM model has produced 26,000+ reasoning tokens without converging on a task it solves in ~500 tokens with thinking disabled. Reasoning is disabled server-side, and every run is checked for reasoning output and for an implausible output-token count.

5. Results

All figures and tables below are generated from `results/final.jsonl`. Cells are labelled *scaffolding*/*model*; the two arms are CLI (blue) and MCP (orange) throughout, in fixed order regardless of sort.

5.1 Results table

60 cells: **51 live**, **9 void**. Void cells carry their reason; a void cell is a claim about what a scaffolding cannot do, not a zero.

Scaffolding	Model	Arm	Init tok	Total in	Cache %	Tools	Tool ok	Done %	Cost $	Wall s
claude-code	fable-5	CLI	22,803	66,696	88.7	5	1.0	100.0	0.7192	21.9
claude-code	fable-5	MCP	28,508	131,094	89.9	9	1.0	100.0	1.4301	33.6
claude-code	opus-5	CLI	23,459	66,676	87.7	4	1.0	100.0	0.3544	20.1
claude-code	opus-5	MCP	26,671	150,827	92.4	8	1.0	100.0	0.8010	32.4
claude-code	sonnet-5	CLI	33,727	99,825	89.9	6	1.0	100.0	0.2100	12.6
claude-code	sonnet-5	MCP	45,462	349,309	93.8	14	1.0	80.0	0.7258	41.5
codex	glm-5.2	CLI	39,284	39,284	80.1	6	1.0	100.0	0.0481	104.7
codex	glm-5.2	MCP	59,293	59,293	97.4	12	1.0	80.0	0.0719	56.3
codex	gpt-5-mini	CLI	121,552	121,552	79.2	14	1.0	80.0	0.0327	37.6
codex	gpt-5-mini	MCP	64,907	64,907	0.0	0	—	80.0	0.0203	52.5
codex	gpt-5.1	CLI	61,169	61,169	44.2	2	1.0	100.0	0.0855	28.1
codex	gpt-5.1	MCP	354,000	354,000	78.8	28	1.0	100.0	0.4560	90.2
codex	gpt-5.2	CLI	160,294	160,294	91.0	22	1.0	100.0	0.2929	31.8
codex	gpt-5.2	MCP	43,127	43,127	64.4	2	1.0	100.0	0.0894	19.3
codex	gpt-5.6-luna	CLI	25,046	25,046	65.0	2	1.0	100.0	0.0028	11.1
codex	gpt-5.6-luna	MCP	18,140	18,140	47.4	2	1.0	100.0	0.0021	11.8
codex	gpt-5.6-sol	CLI	28,631	28,631	65.6	4	1.0	100.0	0.1585	11.6
codex	gpt-5.6-sol	MCP	212,213	212,213	88.1	38	1.0	100.0	1.0981	52.1
codex	gpt-5.6-terra	CLI	28,426	28,426	65.7	4	1.0	100.0	0.0311	7.5
codex	gpt-5.6-terra	MCP	30,997	30,997	65.7	4	1.0	100.0	0.0340	9.8
[illegible] code	[illegible]	MCP	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

5.2 Token cost

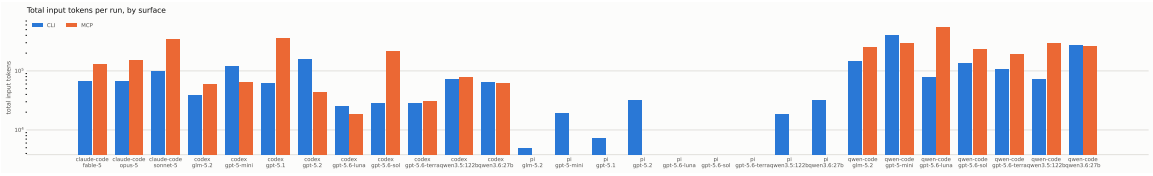

Figure 1: Total input tokens per run, by surface

Cell	CLI tokens	MCP tokens	MCP/CLI
codex/gpt-5.6-sol	28,631	212,213	7.41×
qwen-code/gpt-5.6-luna	77,984	555,763	7.13×
codex/gpt-5.1	61,169	354,000	5.79×
qwen-code/qwen3.5:122b	73,324	289,545	3.95×
claude-code/sonnet-5	99,825	349,309	3.50×
claude-code/opus-5	66,676	150,827	2.26×
claude-code/fable-5	66,696	131,094	1.97×
qwen-code/gpt-5.6-terra	106,225	189,434	1.78×
qwen-code/glm-5.2	142,889	247,723	1.73×
qwen-code/gpt-5.6-sol	133,654	228,894	1.71×
codex/glm-5.2	39,284	59,293	1.51×
codex/gpt-5.6-terra	28,426	30,997	1.09×
codex/qwen3.5:122b	71,507	77,622	1.09×
codex/qwen3.6:27b	63,405	61,228	0.97×
qwen-code/qwen3.6:27b	271,545	260,276	0.96×
codex/gpt-5.6-luna	25,046	18,140	0.72×
qwen-code/gpt-5-mini	405,321	291,964	0.72×
codex/gpt-5-mini	121,552	64,907	0.53×
codex/gpt-5.2	160,294	43,127	0.27×

19 comparable cells · median **1.71×** · range 0.27×–7.41×

The spread is the result. A practitioner reading any single published figure — 1.2×, 35×, or anything between — is reading one cell of this table.

5.3 Tool calls and the tool register

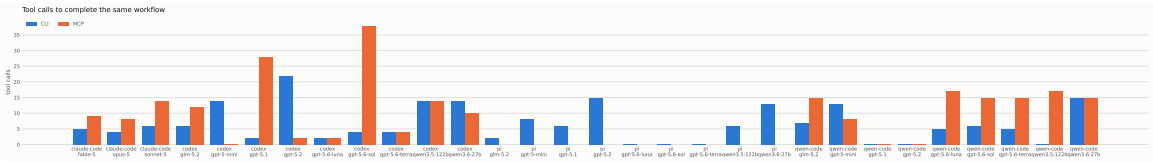

Figure 2: Tool calls to complete the same workflow

Model	Scaffolding	CLI tokens	Calls	Done %
fable-5	claude-code	66,696	5	100.0
glm-5.2	pi	4,843	2	100.0
glm-5.2	codex	39,284	6	100.0
glm-5.2	qwen-code	142,889	7	100.0
gpt-5-mini	pi	19,089	8	100.0
gpt-5-mini	codex	121,552	14	80.0
gpt-5-mini	qwen-code	405,321	13	20.0
gpt-5.1	pi	7,355	6	100.0
gpt-5.1	codex	61,169	2	100.0
gpt-5.2	pi	31,304	15	100.0
gpt-5.2	codex	160,294	22	100.0
gpt-5.6-luna	codex	25,046	2	100.0
gpt-5.6-luna	qwen-code	77,984	5	100.0
gpt-5.6-sol	codex	28,631	4	100.0
gpt-5.6-sol	qwen-code	133,654	6	100.0
gpt-5.6-terra	codex	28,426	4	100.0
gpt-5.6-terra	qwen-code	106,225	5	100.0
opus-5	claude-code	66,676	4	100.0
qwen3.5:122b	pi	18,262	6	100.0
qwen3.5:122b	codex	71,507	14	100.0
qwen3.5:122b	qwen-code	73,324	0	0.0
qwen3.6:27b	pi	31,758	13	100.0
qwen3.6:27b	codex	63,405	14	100.0
qwen3.6:27b	qwen-code	271,545	15	80.0
sonnet-5	claude-code	99,825	6	100.0

Which tools were actually called, aggregated by arm:

Arm	Tool	Calls
CLI	bash	45
CLI	web_fetch	36
CLI	shell:git	24
CLI	shell:aawd-e1-fixture	18
CLI	Bash	15
CLI	run_shell_command	14
CLI	shell:curl	12
CLI	shell:ls	8
CLI	shell:head	6
CLI	read	5
CLI	shell:find	4
CLI	shell:(	2
MCP	mcp__github__get_file_contents	80
MCP	mcp__get_file_contents	56
MCP	shell:gh	12
MCP	Bash	11
MCP	mcp__github__list_tags	10
MCP	shell:git	8
MCP	shell:curl	8
MCP	run_shell_command	8
MCP	ToolSearch	6
MCP	mcp__list_tags	6
MCP	tool_search	6
MCP	mcp__github__search_repositories	4

5.4 Task completion

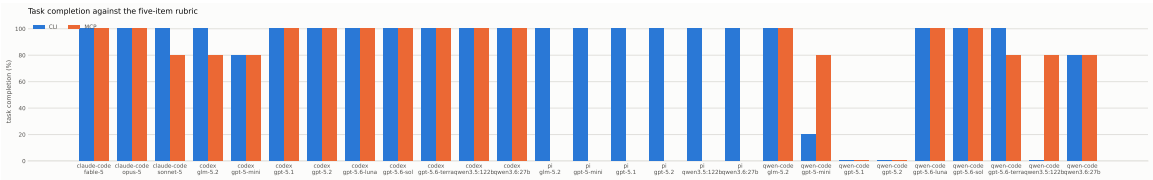

Figure 3: Task completion against the five-item rubric

Arm	Scored runs	Fully complete	Rate	Mean completion
CLI	27	21	78%	84.4%
MCP	21	12	57%	83.8%

Mean completion is near-identical between arms while the rate of *fully* completed runs is not: the MCP arm does not produce worse answers so much as more partial ones.

5.5 Cost

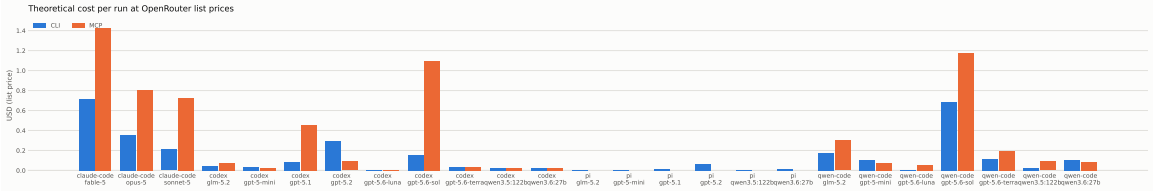

Figure 4: Theoretical cost per run at list prices

Theoretical cost across all scored runs totals **$10.05** at list prices. The five most expensive individual runs:

Cell	Arm	Cost
claude-code/fable-5	MCP	$1.4301
qwen-code/gpt-5.6-sol	MCP	$1.1778
codex/gpt-5.6-sol	MCP	$1.0981
claude-code/opus-5	MCP	$0.8010
claude-code/sonnet-5	MCP	$0.7258

This is a modelled figure, not an invoice: one public price list is applied uniformly to every cell including those served on local hardware, because comparing billing arrangements would not compare workloads.

6. Discussion

A ratio is the wrong shape of answer. The practitioner literature argues about whether MCP costs 1.2× or 35×. Our measurement says the question is malformed: across comparable cells the ratio spans roughly an order of magnitude, and the same protocol against the same task can cost more or less than a CLI depending on which scaffolding and which model it runs under. Any single number reported without naming both is describing one cell.

The scaffolding dominates the surface. The largest effect in our data is not MCP versus CLI at all. On an identical model, arm and task, one scaffolding completed the work in a few thousand tokens while another took two orders of magnitude more. That difference exceeds every MCP-versus-CLI ratio we measured. A practitioner choosing how to reduce agent cost should look at the harness before the protocol — which is not what the current discussion is about.

Tool-call count is a poor proxy for cost. Tokens per tool call varied by more than sixfold across cells. A model that makes few, fat calls can cost as much as one making many thin ones, because

each turn re-sends the accumulated conversation and, on the MCP arm, the registry. Reporting call counts without tokens — as several practitioner comparisons do — can invert the ranking.

Structured tools may help weak models and cost strong ones. Where MCP was cheaper, it was on smaller or weaker models, whose CLI arms made more calls and burned more tokens exploring. A registry that tells the model what operations exist can repay its fixed cost by reducing floundering. This is the least certain of our observations and the one most worth testing at depth: it rests on few cells and it cuts directly against the prevailing advice.

Caching does not resolve the difference. Cache-hit rates were high and comparable on both arms, so the registry does land in the cached prefix — and it does not close the gap, because the CLI arm caches equally well. An argument that prompt caching makes MCP overhead irrelevant is not supported here.

The instrument is the contribution. Five of the hazards in §4 were found by producing a plausible wrong answer first, and three of them — a silently unattached server, a scaffolding merging config from outside the run directory, and sub-agent tokens billed to threads the parent cannot see — would have produced clean, publishable, wrong tables. We expect any independent replication to hit at least one of them.

7. Limitations

One run per cell. The matrix is broad and shallow. A single run per cell establishes that an effect is reproducible in principle, not how often it occurs, and no variance is reported. Deepening the matrix is the obvious next version and the harness supports it directly.

One task, in one domain. The workflow is a GitHub task, and the discriminating item rewards a surface that can aggregate server-side. A different domain — one where the tool catalogue offers exactly the operation needed and the shell does not — would plausibly move the result the other way. We publish the task so that claim can be tested rather than argued.

Default configuration is a choice, not a neutral baseline. Running every scaffolding as it ships measures what a practitioner encounters, and it also means a scaffolding with a bloated default tool set is penalised for a decision that a tuned deployment would reverse.

Prices move. The cost column is a snapshot of one public price list on one date, and it is a modelled figure rather than a billed one.

We wrote the CLI-side affordance. Where a skill document is supplied to the CLI arm, we wrote it, and a reviewer is right to ask whether it is a fair match for the tool catalogue it is compared against. It is published in full for exactly that reason.

8. Conclusion

We set out to measure what it costs to give an agent a capability through a tool catalogue rather than through a command line, and found that the question cannot be answered with a number. Across four scaffoldings and nine models the ratio moves by roughly an order of magnitude, and it is not even consistent in sign.

Two things are worth taking away from that. The first is that the variable the field is arguing about is smaller than a variable it is largely ignoring: the scaffolding's own overhead moved our results further than the protocol did. The second is that measuring this is harder than it looks. Configuration leaks between arms, servers attach silently or not at all, approval modes remove tools from

context without saying so, sub-agents spend tokens off-thread, and caches persist across runs — each producing numbers that look fine.

We therefore publish the instrument rather than a verdict: the task, the rubric, the adapters, the hazards, and every raw run. The result we would most like to be contradicted is the one we hold least firmly — that structured tool catalogues can pay for themselves on weaker models — and the benchmark exists so that contradicting it costs an afternoon rather than a research programme.

Data and code availability

Harness, task, rubric, adapters, cost model, fixture pointer and every raw run log are at `github.com/Lamb-Project/mcp-vs-cli-bench`, archived to Zenodo.

Reproducing the numbers requires four conditions that change results if skipped, each documented in the repository README: reasoning disabled server-side; the fixture pinned at its tag; the pinned `fastapi` version if the LiteLLM proxy is used; and the local KV cache cleared between runs.

References

[Anthropic 2026] Anthropic. *Code execution with MCP: building more efficient agents.* Engineering blog, 2026.

[MCP 2025] Model Context Protocol specification. `modelcontextprotocol.io`, 2025.

[MindStudio 2026] MindStudio. *CLI vs MCP: a controlled comparison of token cost and task reliability.* 2026.

[Reinhard 2026] Reinhard, A. *The hidden cost of MCP servers in your context window.* 2026.

[Vensas 2026] Vensas. *Measuring MCP overhead against the GitHub CLI.* 2026.

[Zechner 2026] Zechner, M. *What I learned building an opinionated and minimal coding agent.* 2026.

> [!note] Marc Reference metadata is from the AAWD reference list and needs a URL-liveness pass before submission — the practitioner citations are the ones most likely to have moved.